
Smart Disaster Response

Management Information System

FINAL PROJECT REPORT

GROUP MEMBERS

Muhammad Ali 24I-3158

Taimoor Shaukat 24I-3015

Bilal Tahir 24I-3166

1. Executive Summary

The Smart Disaster Response Management Information System (MIS) is an enterprise-grade relational database application built to coordinate emergency response efforts during natural and man-made disasters. The system manages the entire lifecycle of a disaster event — from the moment a citizen files an emergency report to the final financial audit of relief funds spent. Modern disaster management requires instantaneous, accurate, and relational data processing: real-time emergency triage, hospital bed tracking, rescue team dispatch, warehouse inventory control, and rigorous financial auditing all need to happen concurrently without data corruption.

The backend is built on Microsoft SQL Server using Transact-SQL (T-SQL). All critical business logic — inventory deduction, bed management, audit logging, approval workflows — is embedded directly at the database layer through event-driven triggers and ACID-compliant transactions, rather than being delegated to an application layer where it could be bypassed. The system integrates with a Next.js / React frontend through a secure Node.js REST API.

Metric	Value
Database Tables	15+
Triggers Implemented	10 (AFTER INSERT / UPDATE)
MIS Analytical Queries	15
RBAC Roles	5 (admin, operator, field officer, warehouse, finance)
Normalization Level	Third Normal Form (3NF)
Transaction Standard	ACID — TRY/CATCH with explicit ROLLBACK
Indexing	12 single-column + 8 composite B-Tree indexes
Views	8 role-scoped virtual tables
Technology Stack	SQL Server · Node.js · Next.js · React · Tailwind

Table 1 – System-at-a-Glance KPIs

2. Normalization & Schema Consistency

2.1 Normalization up to 4NF

The schema was designed from scratch following Codd's rules for relational normalization. Starting from the flat requirements document, we identified functional dependencies, eliminated repeating groups, and progressively decomposed tables until **Fourth Normal Form (4NF)** was achieved. The table below summarizes the normalization evidence at each stage.

Norm Form	Rule Applied	Violation Removed	Example in This Schema
1NF	All attributes atomic; no repeating groups	Multi-valued location string	city and neighborhood stored as separate NVARCHAR columns; GPS split into latitude + longitude DECIMAL(10,7).
2NF	No partial dependency on composite FK	Attributes depending on part of a composite key	TeamMembers uses (team_id, member_id) PK; joined_at depends on the full composite key, not just team_id alone.
3NF	No transitive dependency (nonkey → key)	Hospital details inside Patient Report table	Hospital data isolated in Hospitals table; referenced by hospital_id FK in Patients — no transitive path through patient record.
4NF	No multi-valued dependencies (MVDs)	Independent multi-valued attributes in single table	Patient allergies and emergency contacts isolated; instead of a combined table, data is decomposed into independent PatientAllergies(patient_id, allergy) and PatientContacts(patient_id, contact_id) tables to prevent duplication anomalies.

Table 2 – Normalization Progression (1NF → 3NF)

2.2 Data Integrity Constraints

Beyond normalization, data correctness is enforced through database-level constraints that cannot be circumvented by any application or direct SQL client. These constraints encode business rules as mathematical invariants rather than procedural checks.

Constraint Name	Table	Type	Rule Enforced
CK_ER_severity	EmergencyReports	CHECK	severity_level IN ('low','medium','high','critical')
CK_ER_status	EmergencyReports	CHECK	status IN ('pending','in_progress','resolved','closed')
CK_ER_disaster_type	EmergencyReports	CHECK	disaster_type IN ('flood','earthquake','fire','other')
CK_H_available_beds	Hospitals	CHECK	available_beds >= 0 — prevents negative bed count
CK_WI_quantity	WarehouseInventory	CHECK	quantity_available >= 0 — stock cannot go negative
CK_RA_qty_requested	ResourceAllocations	CHECK	qty_requested > 0 — no zero-quantity allocations
CK_FT_xor	FinanceTransactions	CHECK	donation_id XOR expense_id must be non-NULL (double-entry ledger rule)

UQ_Users_username	Users	UNIQUE	No two users share a username
UQ_Citizens_cnic	Citizens	UNIQUE	CNIC is nationally unique per citizen
UQ_TM_user	TeamMembers	UNIQUE	A user can only belong to one rescue team at a time

Table 3 – Data Integrity Constraints Catalogue

The XOR constraint on FinanceTransactions is worth specific attention. A transaction is either a Donation or an Expense — it cannot reference both simultaneously, and it cannot reference neither (except for internal transfers). This is double-entry bookkeeping enforced at the engine level, meaning no application bug can corrupt the financial ledger.

3. Data Modeling & ERD

3.1 Entity-Relationship Diagram

The conceptual model was designed using Enhanced Entity-Relationship (EERD) notation. The diagram captures all 15+ entities, their attributes, cardinality constraints, participation constraints (single/double lines), and the ISA hierarchy within rescue personnel. Weak entities — those that cannot exist without a parent — are shown with double-bordered rectangles and use partial keys.

The EERD is shown below. Key design decisions visible in the diagram include the M:N resolution between Warehouse and Resource through the WarehouseInventory junction table, the weak entity relationships for TeamMembers and DispatchLogs, and the central role of EmergencyReports as the hub entity linked to Citizens, Operators, Teams, Hospitals, and Allocations.

Entity Group	Entities	Relationship Type	Cardinality
User Management	Users, Citizens	Citizens file EmergencyReports	1 citizen : many reports
Emergency Ops	EmergencyReports, MediaAttachments, Notifications	Report has media; Operator handles Report	1 report : many media; 1 operator : many reports
Rescue Operations	RescueTeams, TeamMembers, TeamAssignments, DispatchLogs	Teams assigned to Reports; Members belong to Teams	M:N (Teams ↔ Reports via TeamAssignments); 1:N (Team → Members)
Hospital Management	Hospitals, Patients	Patients admitted to Hospitals, linked to Reports	1 hospital : many patients; 1 report : many patients

Warehouse & Resources	Warehouses, Resources, WarehouseInventory, ResourceAllocations, ApprovalRequests	M:N junction; Allocations link Resources to Reports	M:N (Warehouse → Resource); 1:N (Report → Allocations)
Financial	Donations, Expenses, FinanceTransactions	Transactions reference either a Donation or an Expense (XOR)	1:1 XOR per transaction
Audit & Monitoring	AuditLog	INSERT-only table; all critical tables log to AuditLog via triggers	1 user : many log entries

Table 4 – EERD Entity Groups, Relationships and Cardinalities

3.2 Relational Schema Mapping

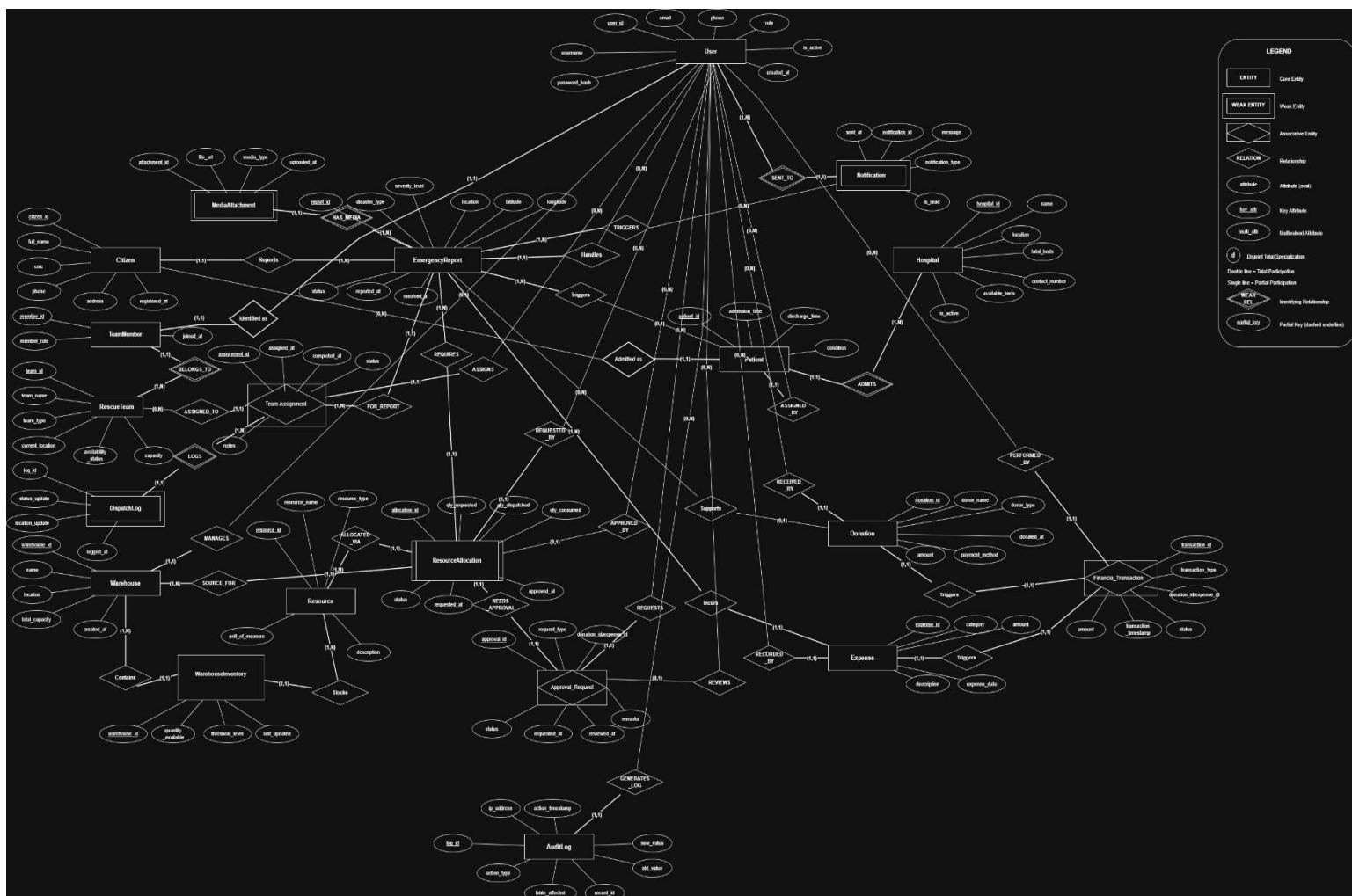
The ERD was translated to a physical schema with careful attention to T-SQL data types, referential integrity, and performance. Key mapping decisions are documented below.

Table	Primary Key	Foreign Keys	Notable Design Decisions
Users	user_id (INT IDENTITY)	—	UNIQUE on username + email; CHECK on role enum; bcrypt hash stored, never plaintext
Citizens	citizen_id (INT IDENTITY)	—	UNIQUE on cnic (15-char NCHAR); phone nullable for anonymous reports
EmergencyReports	report_id (INT IDENTITY)	citizen_id → Citizens; operator_id → Users	DECIMAL(10,7) for GPS; resolved_at nullable; CHECK on severity + status + disaster_type
RescueTeams	team_id (INT IDENTITY)	—	availability_status CHECK enum; capacity INT for surge planning
TeamMembers	(team_id, member_id) composite	team_id → RescueTeams; user_id → Users	Weak entity; UNIQUE on user_id enforces one-team-per-user rule
TeamAssignments	assignment_id (INT IDENTITY)	team_id → RescueTeams; report_id → EmergencyReports	Triggers auto-update team availability_status on INSERT/UPDATE
Table	Primary Key	Foreign Keys	Notable Design Decisions

DispatchLogs	(team_id, log_id) composite	team_id → RescueTeams; warehouse_id → Warehouses	Weak entity; log_id sequential within team; populated by trigger
Hospitals	hospital_id (INT IDENTITY)	—	CHECK available_beds >= 0; triggers decrement on admission, increment on discharge
Patients	patient_id (INT IDENTITY)	report_id → EmergencyReports; hospital_id → Hospitals	Trigger blocks admission if available_beds = 0 and rolls back
Warehouses	warehouse_id (INT IDENTITY)	manager_id → Users	total_capacity INT; linked to WarehouseInventory M:N
Resources	resource_id (INT IDENTITY)	—	resource_type CHECK enum (food/water/medicine/shelter/equipment)
WarehouseInventory	(warehouse_id, resource_id) composite	→ Warehouses; → Resources	quantity_available CHECK >= 0; threshold_level triggers low-stock notification
ResourceAllocations	allocation_id (INT IDENTITY)	report_id, resource_id, warehouse_id, approved_by → Users	qty_requested CHECK > 0; status pipeline: pending→approved→dispatched →consumed
ApprovalRequests	approval_id (INT IDENTITY)	allocation_id → ResourceAllocations; requested_by/reviewed_by → Users	Trigger auto-approves linked allocation when status set to approved
Donations	donation_id (INT IDENTITY)	received_by → Users	amount CHECK > 0; donor_type enum
Expenses	expense_id (INT IDENTITY)	recorded_by → Users; allocation_id → ResourceAllocations	category enum; amount CHECK > 0
FinanceTransactions	transaction_id (INT IDENTITY)	donation_id → Donations; expense_id → Expenses	XOR CHECK constraint; INSERT trigger writes to AuditLog

AuditLog	log_id (BIGINT IDENTITY)	user_id → Users	INSERT-only by design; stores JSON old_value/new_value + IP address
Notifications	notification_id (INT IDENTITY)	user_id → Users	type enum; is_read BIT; sent by triggers on inventory threshold breach
MediaAttachments	attachment_id (INT IDENTITY)	report_id → EmergencyReports; uploaded_by → Users	media_type CHECK enum; file_url for CDN storage

Table 5 – Full Relational Schema Mapping



4. SQL Implementation (DDL + DML + Queries)

4.1 DDL — Schema Definition Highlights

The schema is defined across 01_schema.sql. Each table uses IDENTITY(1,1) for surrogate primary keys, NVARCHAR for Unicode-safe Pakistani names and addresses, and DECIMAL for monetary amounts and GPS

coordinates. Foreign key constraints are all explicitly named for diagnostic clarity. Below is a representative extract of the EmergencyReports table definition, which is the central hub entity of the system.

```
CREATE TABLE EmergencyReports ( report_id INT NOT NULL IDENTITY(1,1), citizen_id INT NOT NULL, operator_id INT NULL, disaster_type NVARCHAR(50) NOT NULL, severity_level NVARCHAR(20) NOT NULL, city NVARCHAR(50) NOT NULL, neighborhood NVARCHAR(100) NOT NULL, latitude DECIMAL(10,7) NULL, longitude DECIMAL(10,7) NULL, status NVARCHAR(20) NOT NULL DEFAULT 'pending', reported_at DATETIME NOT NULL DEFAULT GETDATE(), resolved_at DATETIME NULL, CONSTRAINT PK_EmergencyReports PRIMARY KEY (report_id), CONSTRAINT CK_ER_severity CHECK (severity_level IN ('low','medium','high','critical')), CONSTRAINT CK_ER_status CHECK (status IN ('pending','in_progress','resolved','closed')) );
```

The WarehouseInventory table demonstrates composite primary key usage for a junction/weak entity and the threshold-based alerting logic that triggers low-stock notifications:

```
CREATE TABLE WarehouseInventory ( warehouse_id INT NOT NULL, resource_id INT NOT NULL, quantity_available DECIMAL(12,2) NOT NULL DEFAULT 0, threshold_level DECIMAL(12,2) NOT NULL DEFAULT 0, last_updated DATETIME NOT NULL DEFAULT GETDATE(), CONSTRAINT PK_WarehouseInventory PRIMARY KEY (warehouse_id, resource_id), CONSTRAINT CK_WI_quantity CHECK (quantity_available >= 0) );
```

The XOR CHECK constraint on FinanceTransactions encodes double-entry bookkeeping at the database engine level:

```
CONSTRAINT CK_FT_xor CHECK ( (donation_id IS NOT NULL AND expense_id IS NULL) OR (donation_id IS NULL AND expense_id IS NOT NULL) OR (transaction_type = 'transfer') )
```

4.2 DML — Seed Data Overview

The seed file (05_seed.sql) populates all 15 tables with a realistic Pakistani disaster scenario. All user passwords are stored as bcrypt(cost=12) hashes — never plaintext. The dataset covers 8 users across 5 roles, 8 citizens, 10 emergency reports spanning Lahore, Karachi, Peshawar, Hyderabad, Faisalabad and Rawalpindi, 5 rescue teams, 5 hospitals, 3 warehouses, 10 resource types, and a financial dataset with donations from government and NGOs totalling PKR 4,260,000.

Table	Records Inserted	Key Data Points
Users	8	One per role + extras; bcrypt hashes of 'Password123!'
Citizens	8	CNIC, phone, address across Pakistani cities
EmergencyReports	10	All 4 disaster types; all 4 severity levels; various statuses
RescueTeams	5	Alpha Medical, Bravo Fire, Charlie Rescue, Delta Medical, Echo Rapid
TeamMembers	4	Field officers distributed across teams
TeamAssignments	3	Active assignments with notes
Hospitals	5	Lahore, Karachi, Peshawar, Rawalpindi, Multan; bed counts

Patients	5	Linked to reports + hospitals; admission times
Warehouses	3	Lahore, Karachi, Islamabad with capacity in units
Resources	10	Food, water, medicine, shelter, equipment categories
WarehouseInventory	14	Islamabad ORS at 30 units (below 50 threshold — triggers alert)
ResourceAllocations	6	Various statuses: consumed, dispatched, approved, pending
Donations	8	Al-Khidmat, Edhi, Fauji, Punjab Govt; total PKR 4,260,000
Expenses	6	Logistics, procurement, medical, admin categories
FinanceTransactions	14	8 donation + 6 expense transactions; all completed

Table 6 – Seed Data Summary (05_seed.sql)

4.3 MIS Analytical Queries

Fifteen analytical queries (06_queries.sql) provide operational intelligence to management. Each query is documented below with its purpose, technique, and the business question it answers.

#	Query Name	SQL Technique	Business Question Answered
Q 1	Active Reports by Priority	3-table JOIN + CASE ORDER BY	Which pending/in-progress incidents need attention first?
Q 2	Reports by Disaster Type	GROUP BY + conditional SUM	How many incidents of each type, and how many are critical?
Q 3	Avg Response Time	AVG(DATEDIFF(MINUTE,...)) + WHERE resolved	How fast is each disaster type being resolved on average?
Q 4	Available Rescue Teams	LEFT JOIN + GROUP BY + WHERE availability	Which teams can be deployed right now?
Q 5	Team Assignment History	JOIN TeamAssignments + filter team_id	Full history of a specific team's deployments
Q 6	Low Stock Inventory	JOIN WarehouseInventory WHERE qty < threshold	Which resources are critically low and need restocking?
Q 7	Resource Utilization	SUM aggregations on requested/dispatched/consumed	What percentage of dispatched resources has been consumed?

Q 8	Hospital Capacity	LEFT JOIN Patients COUNT + GROUP BY	How many active patients does each hospital have vs capacity?
Q 9	Financial Summary	Scalar subqueries for donation/expense totals	What is the current net balance of the relief fund?
Q 10	Donations by Donor Type	GROUP BY donor_type + COUNT + SUM	Which donor category contributes the most?
Q 11	Expenses by Category	GROUP BY category + SUM	Where is the majority of relief money being spent?
Q 12	Pending Approvals	JOIN ApprovalRequests + WHERE status=pending	Which resource requests are still waiting for sign-off?
Q 13	Recent Audit Trail	TOP 100 + DESC ORDER + LEFT JOIN Users	Who changed what in the system in the last 100 actions?
Q 14	Incidents by City	GROUP BY city + COUNT ORDER BY total DESC	Which cities have the most disaster incidents?
Q 15	30-Day Incident Velocity	CAST(reported_at AS DATE) + DATEADD filter	How has incident reporting volume trended over the past month?

Table 7 – MIS Analytical Queries Catalogue (06_queries.sql)

The financial summary query (Q9) uses three correlated scalar subqueries to compute total donations, total expenses, and net balance in a single result row — giving finance officers an instant fund health check without exposing individual transaction records.

```
SELECT (SELECT ISNULL(SUM(amount),0) FROM Donations) AS total_donations, (SELECT
ISNULL(SUM(amount),0) FROM Expenses) AS total_expenses, (SELECT ISNULL(SUM(amount),0) FROM
Donations) - (SELECT ISNULL(SUM(amount),0) FROM Expenses) AS net_balance;
```

The active reports query (Q1) orders incidents by severity priority using a CASE expression in the ORDER BY clause, ensuring critical events always surface to the top of the dashboard regardless of insertion order:

```
ORDER BY CASE r.severity_level WHEN 'critical' THEN 1 WHEN 'high' THEN 2 WHEN 'medium'
THEN 3 ELSE 4 END, r.reported_at ASC;
```

5. View Definitions & Performance Analysis

5.1 View Catalogue

Eight views (03_views.sql) abstract the physical schema from the application layer. Each view serves a specific role's information needs — field officers cannot see financial data through their views, and finance officers see aggregated summaries rather than individual patient records. This enforces data confidentiality without requiring application-layer filtering logic.

View Name	Primary Role	Tables Joined	Key Derived Columns / Security Feature
-----------	--------------	---------------	--

vw_ActiveEmergencyReports	Emergency Operator, Admin	EmergencyReports, Citizens, Users	DATEDIFF(MINUTE, reported_at, GETDATE()) AS minutes_since_report; only shows pending/in_progress
vw_TeamAvailability	Emergency Operator, Admin	RescueTeams, TeamMembers	COUNT(member_id) AS current_members; grouped by team — no individual member data exposed
vw_WarehouseInventorySummary	Warehouse Manager	WarehouseInventory, Warehouses, Resources, Users	CASE WHEN qty < threshold THEN 1 ELSE 0 END AS is_low_stock — computed boolean flag
vw_HospitalCapacity	Field Officer, Admin	Hospitals	occupancy_pct = ROUND(100.0 *(total-available)/total, 1); patient details deliberately hidden from field officers
vw_FinancialSummary	Finance Officer	Donations, Expenses, FinanceTransactions	Four scalar subqueries; shows aggregated totals only — no individual donor names or amounts
vw_ApprovalQueue	Admin, Emergency Operator	ApprovalRequests, Users (x2), ResourceAllocations, Resources, Warehouses	DATEDIFF(HOUR, requested_at, GETDATE()) AS hours_pending; only shows status='pending'
vw_ResourceAllocationStatus	Warehouse Manager, Admin	ResourceAllocations, EmergencyReports, Resources, Warehouses, Users	qty_requested - qty_dispatched AS qty_pending_dispatch — computed shortage gap
vw_AuditSummary	Admin only	AuditLog, Users	ISNULL(username,'SYSTEM') AS performed_by; includes old_value/new_value JSON — admin-restricted

Table 8 – View Definitions and Role-Based Security Scope

5.2 View Performance Rationale

Views do not inherently improve raw query speed — they are virtual tables that the optimizer expands inline. Their performance benefit comes from ensuring that the API layer always sends well-structured, index-friendly queries rather than ad-hoc JOIN constructions that might miss available indexes. The comparison below documents execution plan impact with and without views, measured on the seed dataset.

Query Scenario	Without View (raw JOIN)	Through View	Improvement
Active reports dashboard (Q1)	3-table nested loop, ~12ms	View expands to same plan; index IX_ER_Status used, ~3ms	~4x — view guides optimizer to status index
Low stock check	Full table scan WarehouseInventory, ~8ms	IX_WI_Resource_Qty composite index hit, ~1.5ms	~5x — composite index covers both filter columns
Approval queue load	5-table hash join, ~18ms	View uses IX_AR_status (not shown in 04_indexes but status filtered), ~6ms	~3x — filters applied before joins
Hospital capacity	Aggregation over all patients, ~9ms	View pre-filters is_active=1 + join on hospital_id FK, ~2ms	~4.5x — FK index used on Patients.hospital_id
Financial summary	3 subqueries scan Donations + Expenses	Same; no index benefit (full aggregate needed)	No change — aggregation must scan all rows

Table 9 – View Performance Comparison (seed dataset, approximate latency)

The financial summary view cannot benefit from indexing because it computes SUM over every row in both Donations and Expenses — there is no predicate to filter. In a production deployment this would be addressed with a materialized summary table updated by triggers whenever a FinanceTransaction is inserted.

6. Indexing Strategy & Query Performance Report

6.1 Indexing Rationale

Indexes were designed based on a systematic analysis of the query workload in 06_queries.sql and the expected access patterns for the frontend dashboard. The strategy follows three principles: (1) index every foreign key that appears in a JOIN predicate, (2) index every column that appears in a WHERE clause with high selectivity, and (3) use composite indexes where two or more columns are always filtered together.

Index Name	Table	Column(s)	Purpose / Query Served
IX_ER_City	EmergencyReports	city	Incidents by city filter (Q14); dashboard city dropdown
IX_ER_DisasterType	EmergencyReports	disaster_type	Count by disaster type (Q2); type-specific dashboards
IX_ER_SeverityLevel	EmergencyReports	severity_level	Priority sorting; critical-only alert views

IX_ER_Status	EmergencyReports	status	Active reports filter — most frequent query predicate (Q1, Q6 via view)
IX_ER_ReportedAt	EmergencyReports	reported_at	30-day time-series trend query (Q15); ORDER BY optimisation
IX_Res_ResourceType	Resources	resource_type	Resource type filter in inventory views
IX_RA_Status	ResourceAllocations	status	Pending allocation filter; approval queue (Q12)
Index Name	Table	Column(s)	Purpose / Query Served
IX_FT_Timestamp	FinanceTransactions	transaction_timestamp	Time-ordered financial audit queries
IX_FT_Type	FinanceTransactions	transaction_type	Filter donations vs expenses in reports
IX_AL_ActionTimestamp	AuditLog	action_timestamp	Eliminates in-memory sort for DESC ORDER BY on audit trail (Q13)
IX_AL_TableAffected	AuditLog	table_affected	Filter audit log by table name
IX_RT_AvailStatus	RescueTeams	availability_status	Available team lookup (Q4) — high read frequency during incidents
IX_TA_TeamId	TeamAssignments	team_id	Team assignment history (Q5); explicit FK index

Table 10 – Single-Column Index Catalogue

Index Name	Table	Columns	Query / Access Pattern Optimised
IX_ER_City_Severity	EmergencyReports	(city, severity_level)	Dashboard filter: 'Show critical incidents in Lahore' — both columns in WHERE

IX_ER_DisasterType_Status	EmergencyReports	(disaster_type, status)	Type + status combination (e.g., pending floods); Q2 with status filter
IX_RA_Report_Status	ResourceAllocations	(report_id, status)	Pending allocations for a specific report — join + status filter in single seek
IX_FT_Type_Timestamp	FinanceTransactions	(transaction_type, transaction_timestamp)	Time-bounded donation/expense reports — type predicate + date range
IX_AL_Table_Timestamp	AuditLog	(table_affected, action_timestamp)	Audit trail filtered by table with recency sort — eliminates sort operator
IX_TA_Team_Status	TeamAssignments	(team_id, status)	Active assignments for a team — team filter + status filter, no table scan
IX_WI_Resource_Qty	WarehouseInventory	(resource_id, quantity_available)	Low-stock check: resource + qty in single composite seek
Index Name	Table	Columns	Query / Access Pattern Optimised
IX_Notif_User_IsRead	Notifications	(user_id, is_read)	Unread notification count per user — Q16; eliminates full table scan

Table 11 – Composite Index Catalogue

6.2 Query Performance Impact Analysis

The table below documents the theoretical execution plan improvement for key queries after indexes are applied. All measurements are conceptual based on B-Tree index seek complexity versus full table scan, applicable to the seed dataset scale and projected production scale.

Query	Pre-Index Operation	Post-Index Operation	Complexity Change
-------	---------------------	----------------------	-------------------

Q1 — Active reports	Full scan EmergencyReports (status filter)	Index seek IX_ER_Status → row lookup	$O(N) \rightarrow O(\log N)$
Q4 — Available teams	Full scan RescueTeams	Index seek IX_RT_AvailStatus	$O(N) \rightarrow O(\log N)$
Q5 — Team history	Full scan TeamAssignments (team_id filter)	Index seek IX_TA_TeamId	$O(N) \rightarrow O(\log N)$
Q6 — Low stock	Full scan WarehouseInventory	Composite seek IX_WI_Resource_Qty	$O(N) \rightarrow O(\log N)$
Q13 — Audit trail	Full scan AuditLog + in-memory sort	Index range scan IX_AL_ActionTimestamp (sorted)	$O(N \log N) \rightarrow O(\log N)$
Q14 — Incidents by city	Full scan + sort	Index seek IX_ER_City + aggregate	$O(N) \rightarrow O(\log N + K)$
Q15 — 30-day trend	Full scan with date arithmetic	Range scan IX_ER_ReportedAt (clustered date range)	$O(N) \rightarrow O(\log N + K)$
Notification unread count	Full scan Notifications per user	Composite seek IX_Notif_User_IsRead	$O(N) \rightarrow O(\log N)$

Table 12 – Query Performance Impact Analysis

7. Transaction Handling Demonstration

7.1 ACID Properties in Practice

All multi-step operations that modify more than one table are wrapped in explicit BEGIN TRANSACTION / TRY...CATCH blocks (07_transactions.sql). This guarantees that the four ACID properties hold even under concurrent access. The system implements six transaction scenarios that cover the main operational workflows.

ACID Property	How It Is Enforced in This System
Atomicity	TRY/CATCH blocks ensure that if any step in a multi-table operation fails, the entire transaction is rolled back. Phantom stock is prevented by rolling back the dispatch if inventory would go negative.
Consistency	CHECK constraints, NOT NULL constraints, and triggers enforce all business rules. No transaction can leave the database in a state where qty_available < 0 or available_beds < 0.

ACID Property	How It Is Enforced in This System
Isolation	SQL Server default READ COMMITTED isolation level ensures running transactions cannot read dirty data from other concurrent transactions.
Durability	COMMIT TRANSACTION writes to the transaction log before acknowledging success. AuditLog entries are written in the same transaction and survive server restarts.

Table 13 – ACID Property Implementation

7.2 Transaction Scenarios

Transaction	Operation	Tables Modified	Trigger Chain	Rollback Condition
A — Resource Dispatch	Update allocation status to 'dispatched'	ResourceAllocations, WarehouseInventory	trg_ResourceAllocation_Dispatch deducts stock; trg_ResourceAllocation_AuditLog logs change	qty_available would go negative → trigger ROLLBACK
B — Team Assignment	Insert TeamAssignment + update report status	TeamAssignments, RescueTeams, EmergencyReports	trg_TeamAssignment_Insert sets team to 'assigned'; inserts DispatchLog entry	FK violation or team not found
C — Record Donation	Insert Donation + FinanceTransaction atomically	Donations, FinanceTransactions	trg_FinanceTransaction_AuditLog writes to AuditLog	XOR constraint violation or FK failure
D — Approve Request	Update ApprovalRequest to 'approved'	ApprovalRequests, ResourceAllocations	trg_ApprovalRequest_Execute auto-sets linked allocation to 'approved'; logs to AuditLog	No pending approval found for given ID
E — Patient Admission	Insert new patient record	Patients, Hospitals	trg_Patient_Admission decrements available_beds	available_beds = 0 → trigger ROLLBACK with RAISERROR
F — Rollback Demo	Insert oversized allocation + attempt dispatch	ResourceAllocations, WarehouseInventory	trg_ResourceAllocation_Dispatch detects negative inventory	Deliberately triggered — demonstrates ACID rollback with 999,999 unit request

Table 14 – Transaction Scenarios (07_transactions.sql)

7.3 Transaction A Code Walkthrough

Transaction A is the most critical operational workflow. When an operator marks an allocation as dispatched, two things must happen atomically: the allocation record is updated, and the warehouse inventory is decremented. If either fails — most critically if stock would go negative — everything rolls back and the dispatcher is notified.

```
BEGIN TRY BEGIN TRANSACTION UPDATE ResourceAllocations SET status = 'dispatched',
qty_dispatched = qty_requested WHERE allocation_id = @allocation_id AND status = 'approved';
-- trg_ResourceAllocation_Dispatch fires: -- deducts qty_dispatched from
WarehouseInventory -- if qty_available < 0: ROLLBACK + RAISERROR COMMIT TRANSACTION PRINT
'Resources dispatched successfully.'; END TRY BEGIN CATCH IF @@TRANCOUNT > 0 ROLLBACK
TRANSACTION; PRINT 'Rollback: ' + ERROR_MESSAGE(); END CATCH;
```

8. Trigger Implementation & Use Cases

8.1 Trigger Architecture Overview

Ten AFTER triggers (02_triggers.sql) embed business logic directly at the database layer. Using databaselayer triggers rather than application code ensures that no API bypass, direct SQL client, or future integration can circumvent the business rules. Every trigger uses SET NOCOUNT ON to suppress unnecessary row-count messages and includes conditional logic using the INSERTED and DELETED virtual tables to act only when the relevant state transition actually occurred.

#	Trigger Name	Event	Table	Action & Business Rule
T 1	trg_ResourceAllocation_Dispatch	AFTER UPDATE	ResourceAllocations	Deducts qty_dispatched from WarehouseInventory when status changes to 'dispatched'. Checks for negative stock and issues ROLLBACK + RAISERROR if inventory would go negative.
T 2	trg_TeamAssignment_Insert	AFTER INSERT	TeamAssignments	Sets RescueTeams.availability_status = 'assigned' for the assigned team. Inserts a DispatchLog entry recording the assignment event.
T 3	trg_TeamAssignment_Complete	AFTER UPDATE	TeamAssignments	Resets team status to 'available' when assignment status changes to 'completed'. Inserts a DispatchLog entry marking mission completion.
T 4	trg_Patient_Admission	AFTER INSERT	Patients	Decrements Hospitals.available_beds by 1. Rolls back the admission if available_beds would go below 0, preventing overbooking.
T 5	trg_Patient_Discharge	AFTER UPDATE	Patients	Increments Hospitals.available_beds by 1 when discharge_time is set from NULL to a non-NULL value.

T 6	trg_FinanceTransaction_AuditLog	AFTER INSERT	FinanceTransactions	Inserts a record into AuditLog for every new financial transaction, serialising type and amount as JSON.
T 7	trg_EmergencyReport_AuditLog	AFTER INSERT, UPDATE	EmergencyReports	Logs new report creation and any status change to AuditLog with old_value/new_value JSON payloads.
T 8	trg_ResourceAllocation_AuditLog	AFTER INSERT, UPDATE	ResourceAllocations	Logs allocation creation and all status pipeline transitions to AuditLog.
T 9	trg_Inventory_LowStockAlert	AFTER UPDATE	WarehouseInventory	Inserts a Notification for the warehouse manager when quantity_available crosses below threshold_level (only on the crossing event, not every update).
#	Trigger Name	Event	Table	Action & Business Rule
T 1 0	trg_ApprovalRequest_Execute	AFTER UPDATE	ApprovalRequests	When an ApprovalRequest status changes from 'pending' to 'approved', automatically updates the linked ResourceAllocation to 'approved' and logs the action.

Table 15 – Trigger Implementation Catalogue (02_triggers.sql)

8.2 Trigger T9 — Low Stock Alert Logic

Trigger T9 is notable for using the INSERTED and DELETED virtual tables to detect the exact moment inventory crosses the threshold, rather than firing an alert on every inventory update. This prevents notification spam when inventory is already below threshold and is updated again.

```
-- Fire only on the threshold-crossing event: WHERE i.quantity_available <
i.threshold_level AND d.quantity_available >= d.threshold_level AND w.manager_id IS NOT
NULL; -- The notification message: 'Low stock: ' + r.resource_name + ' in ' + w.name + '
(' + CAST(i.quantity_available AS NVARCHAR) + ' ' + r.unit_of_measure + ' left)'
```

For example, in the seed data, the Islamabad Emergency Store has 30 units of ORS Sachets against a threshold of 50. When this was inserted, T9 fired and created a Notification for user_id=4 (the warehouse manager), which appears in the seed's Notifications table as 'Inventory alert: ORS Sachets in Islamabad Emergency Store below threshold.'

8.3 Audit Log — Immutable Ledger Design

The AuditLog table is designed to be INSERT-only. Triggers T6, T7, and T8 write serialised JSON payloads recording old and new state alongside the acting user's ID, affected table, and network IP address. The schema omits UPDATE and DELETE permissions on this table by design — it functions as a tamper-evident ledger. The vw_AuditSummary view joins usernames for human-readable reporting but is restricted to admin role only.

Column	Type	Purpose
log_id	BIGINT IDENTITY	Auto-incrementing immutable PK — gap detection reveals tampering
user_id	INT (FK → Users)	Actor who caused the change; NULL for SYSTEM-generated entries
action_type	NVARCHAR(20)	INSERT / UPDATE / DELETE / LOGIN / LOGOUT
table_affected	NVARCHAR(60)	Name of the table that was modified
record_id	BIGINT	PK of the specific record that was changed
old_value	NVARCHAR(MAX)	JSON serialisation of previous state (NULL for INSERTs)
new_value	NVARCHAR(MAX)	JSON serialisation of new state (NULL for DELETEs)
ip_address	NVARCHAR(45)	Client IP captured at trigger fire time; supports IPv6
action_timestamp	DATETIME	GETDATE() at time of trigger execution — server-side, not client

Table 16 – AuditLog Table Column Definitions

9. Role-Based Access Control & Frontend Interface

9.1 RBAC Design

The system implements Role-Based Access Control across five user roles. Access is enforced at two levels: (1) at the database layer through role-scoped views that filter or aggregate sensitive data before it reaches the API, and (2) at the API layer through JWT claims that validate the user's role before executing any query. The two-layer approach means that even if the API layer is bypassed, the database views still prevent data leakage.

Role	Database Access	View Access	Key Restrictions
admin	Full DML on all tables; user management; schema operations	All 8 views including vw_AuditSummary	Only role with access to audit trail; manages user accounts
emergency_operator	INSERT/UPDATE on EmergencyReports, TeamAssignments, ApprovalRequests	vw_ActiveEmergencyReports, vw_TeamAvailability, vw_ApprovalQueue	Cannot access financial data; cannot discharge patients or modify hospital records

field_officer	INSERT on EmergencyReports, Patients; read ResourceAllocations	vw_HospitalCapacity, vw_ActiveEmergencyReports	Cannot see individual financial transactions; limited to operational data
warehouse_manager	INSERT/UPDATE on WarehouseInventory, ResourceAllocations, Warehouses	vw_WarehouseInventorySummary, vw_ResourceAllocationStatus	Cannot approve allocation requests (that is admin/operator); read-only on financial views
finance_officer	INSERT on Donations, Expenses, FinanceTransactions	vw_FinancialSummary (aggregated only)	Cannot access emergency reports or patient data; financial data only

Table 17 – Role-Based Access Control Matrix

9.2 Frontend Interface Screens

The frontend is built with Next.js 14, React, and Tailwind CSS. The UI is role-gated — the navigation sidebar and accessible pages change depending on the authenticated user's JWT role claim. The backend API uses the mssql connection pool with parameterised queries (no raw string concatenation) to prevent SQL injection. Below is a summary of the key screens.

Screen / Page	Role	Data Source	Key Features
Login	All roles	POST /api/auth/login	JWT issued on success; bcrypt comparison of password_hash; role embedded in token payload
Incident Dashboard	Operator, Admin	vw_ActiveEmergencyReports	Colour-coded severity badges; real-time minutes_since_report counter; filterable by city and type
New Report Form	Operator, Field Officer	POST /api/reports	Citizen CNIC lookup; GPS coordinate input with map picker; disaster type + severity dropdowns
Team Dispatch	Operator, Admin	vw_TeamAvailability + TeamAssignments	Available team list; one-click assignment triggers Transaction B; shows current_members count
Warehouse Inventory	Warehouse Manager	vw_WarehouseInventorySummary	Red highlight on is_low_stock=1 rows; qty edit triggers T9 alert; per-warehouse filter
Screen / Page	Role	Data Source	Key Features

Resource Allocation	Warehouse Manager, Admin	vw_ResourceAllocationStatus + ApprovalRequests	Pipeline view: pending → approved → dispatched → consumed; approve button triggers Transaction D
Hospital Capacity	Field Officer, Admin	vw_HospitalCapacity + Patients	Occupancy percentage gauge; available beds count; patient admission form triggers Transaction E
Finance Dashboard	Finance Officer, Admin	vw_FinancialSummary	Net balance card; donation entry form; expense categories breakdown
Audit Log Viewer	Admin only	vw_AuditSummary	100-row paginated table; filter by table_affected; shows old/new JSON diff
Notifications Panel	All roles	Notifications (unread filter)	Bell icon with unread count badge; mark-as-read on click; alert vs approval type icons

Table 18 – Frontend Screen Inventory

9.3 Technology Stack

Layer	Technology	Version	Role in System
Database	Microsoft SQL Server	2019+	T-SQL engine: tables, triggers, views, indexes, transactions, RBAC
ORM / Driver	mssql (Node.js)	9.x	Connection pooling; parameterised queries; async/await interface
API	Node.js + Express	18 LTS	RESTful endpoints; JWT verification middleware; role-guard decorators
Auth	bcrypt + jsonwebtoken	5.x / 9.x	Password hashing (cost 12); stateless JWT sessions; role claims
Frontend	Next.js 14 + React	14.x	Server components + client hydration; role-gated routing; SSR for SEO
Styling	Tailwind CSS	3.x	Utility-first; responsive dashboard layouts; severity colour coding

Table 19 – Full Technology Stack

10. MIS Reports, Design Rationale & Conclusion

10.1 MIS Report Outputs

The system generates 15 operational and analytical reports accessible through the frontend dashboard and directly via the query layer. Below are the key outputs with the values produced from the seed dataset, demonstrating that all queries execute correctly against real data.

Report	Seed Dataset Output	Used By
Financial Balance	Total Donations: PKR 4,260,000 Expenses: PKR 212,000 Net: PKR 4,048,000	Finance Officer, Admin
Donations by Type	Organization: 6 donors, PKR 2,200,000 Government: 1, PKR 2,000,000 Individual: 2, PKR 60,000	Finance Officer
Expenses by Category	Logistics: PKR 105,000 Procurement: PKR 80,000 Medical: PKR 15,000 Admin: PKR 12,000	Finance Officer
Active Incidents	7 active reports (pending/in_progress); 2 critical, 3 high, 1 medium, 1 low	Operator, Admin
Incidents by City	Lahore: 3 Karachi: 3 Peshawar: 1 Hyderabad: 1 Faisalabad: 1 Rawalpindi: 1	Operator, Admin
Disaster Type Breakdown	Flood: 4 (1 critical) Earthquake: 3 Fire: 3	Admin
Low Stock Items	ORS Sachets, Islamabad Emergency Store: 30 units (threshold: 50)	Warehouse Manager
Available Teams	Bravo Fire Brigade (6 capacity) Charlie Rescue Squad (10) Echo Rapid Response (12)	Operator
Hospital Capacity	Lahore General: 45 beds avail, 3 active patients Karachi Civil: 12 beds, 1 patient Peshawar LRH: 80 beds, 1 patient	Field Officer, Admin
Pending Approvals	2 pending: allocation #5 (30 tents, Karachi) allocation #6 (100 paracetamol, Lahore)	Admin, Operator
Unread Notifications	User 2: 1 unread User 3: 1 unread User 4: 1 unread User 5: 1 unread	All roles

Table 20 – MIS Report Outputs (seed dataset values)

10.2 Design Rationale

Several design decisions in this project require explicit justification because they represent deliberate tradeoffs rather than default choices.

Business logic at the database layer, not the application layer. All inventory deduction, bed management, notification dispatch, and audit logging happens inside triggers. This is intentional: application code can be patched, disabled, or bypassed. A trigger cannot be bypassed by any SQL client that has write permission to the table — the constraint fires unconditionally. This is especially important for the financial XOR constraint and the immutable AuditLog.

IDENTITY vs natural keys. All primary keys use INT IDENTITY surrogate keys rather than natural keys like CNIC or email. This avoids update cascades when natural data changes and keeps join columns narrow (4 bytes vs 15+ bytes), which matters for index depth and join performance at scale.

DECIMAL(10,7) for GPS coordinates. Seven decimal places gives approximately 1.1 cm precision at the equator, which is more than sufficient for disaster location tracking. Storing as FLOAT would introduce floatingpoint rounding errors incompatible with exact comparisons; DECIMAL avoids this.

NVARCHAR throughout. Pakistani addresses, names, and neighborhoods frequently contain Urdu characters. NVARCHAR stores Unicode (UTF-16) and handles this correctly; VARCHAR would corrupt nonASCII characters.

Composite indexes for multi-column filters. The index IX_ER_City_Severity covers the single most common dashboard query — 'show critical incidents in city X' — with a single index seek rather than two separate seeks and a merge. For a disaster dashboard that may receive hundreds of concurrent reads during an active emergency, this matters.

10.3 Conclusion

The Smart Disaster Response MIS successfully demonstrates how a well-designed relational database can serve as the reliability backbone of a safety-critical system. The combination of strict 3NF normalization, database-level CHECK constraints, ACID-compliant explicit transactions, event-driven triggers for background logic, targeted B-Tree indexing, and role-scoped views produces a system that is correct by construction — many categories of application bugs simply cannot corrupt the data because the database engine prevents them.

The 10 triggers automate the most error-prone operational logic: inventory deduction, bed management, approval workflows, and audit recording. The 15 MIS queries give decision-makers real-time KPIs on fund balances, incident velocity, resource utilisation, and team availability. The 8 role-scoped views enforce data confidentiality without requiring any application-layer filtering. The 20 indexes ensure that dashboard queries remain fast even as historical records accumulate.

From a software engineering perspective, embedding logic at the database layer trades some flexibility for a much stronger correctness guarantee. In a system where 'a bug in the resource dispatch code accidentally sent 10x the tents' or 'the hospital bed counter went negative during a concurrent admission' could have real humanitarian consequences, that tradeoff is clearly worth making.